

Прохождение внешнего курса.

Часть 3

Отчет

Гайдук Софья Сергеевна

Содержание

1	Цель работы	6
2	Выполнение лабораторной работы	7
3	Выводы	25
	Список литературы	26

Список иллюстраций

2.1	image1	7
2.2	image2	8
2.3	image3	8
2.4	image4	9
2.5	image5	9
2.6	image6	9
2.7	image7	10
2.8	image8	10
2.9	image9	10
2.10	image10	11
2.11	image11	11
2.12	image12	12
2.13	image13	12
2.14	image14	12
2.15	image15	13
2.16	image16	13
2.17	image17	13
2.18	image18	14
2.19	image19	14
2.20	image20	14
2.21	image21	15
2.22	image22	15
2.23	image23	15
2.24	image24	16
2.25	image25	16
2.26	image26	16
2.27	image27	17
2.28	image28	17
2.29	image29	18
2.30	image30	18
2.31	image31	18
2.32	image32	19
2.33	image33	19
2.34	image34	19
2.35	image35	20
2.36	image36	20

2.37	image37	21
2.38	image38	21
2.39	image39	22
2.40	image40	22
2.41	image41	22
2.42	image42	23
2.43	image43	23
2.44	image44	23
2.45	image45	24

Список таблиц

1 Цель работы

Изучить операционную систему Linux. Прохождение 3 этапа внешнего курса.

2 Выполнение лабораторной работы

: - для работы команды, q - выйти без сохранения. Enter - клавиша для выхода (рис. 2.1).

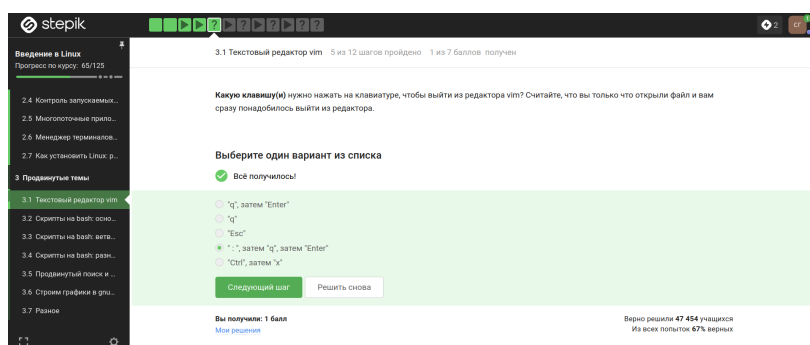


Рисунок 2.1: image1

Словами, кроме привычных нам, также считаются точка, =, !, два пробела, поэтому слов 9. Чтобы попасть в конец строки, действительно, нужно меньше нажатий на W, чем на w. В строке 5 больших слов. Курсор не переместиться на «.», нажимая на W. Действительно, мы окажемся в одном месте после 10 нажатий W и после 10 нажатий w (рис. 2.2).

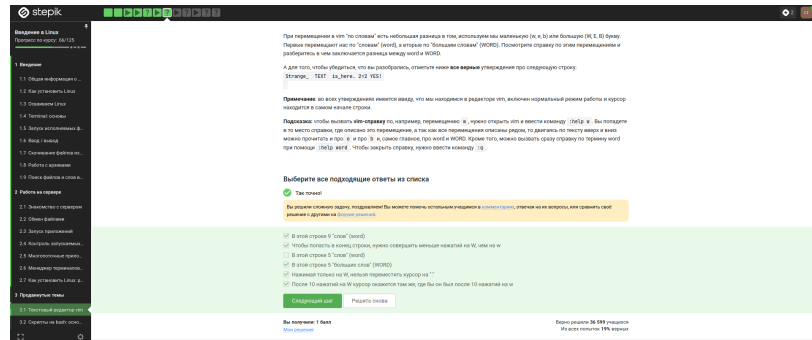


Рисунок 2.2: image2

w — на слово вправо, b — на слово влево, i — начать ввод перед курсором, р — вставка содержимого неименованного буфера под курсором, Р — вставка содержимого неименованного буфера перед курсором, \$ — в конец текущей строки, уу (также Y) — копирование текущей строки в неименованный буфер, уу — копирование числа строк начиная с текущей в неименованный буфер. Поэтому d2w\$bfour four <>, d2wwywPr, xxxxxxxxwywPr (рис. 2.3).

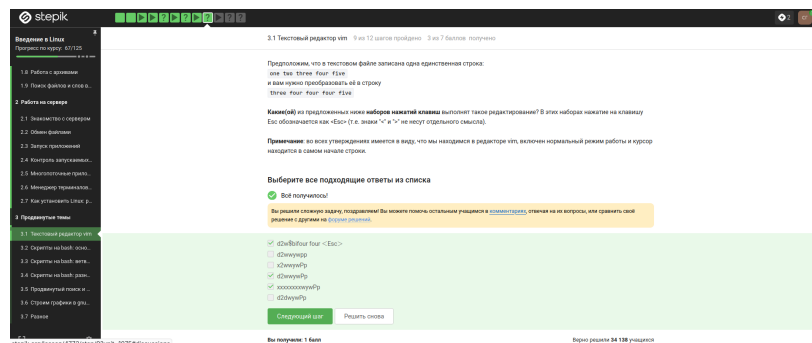


Рисунок 2.3: image3

:(сколько_заменяем)s/что_меняем)(на_что_меняем). Для замены используем % (рис. 2.4).

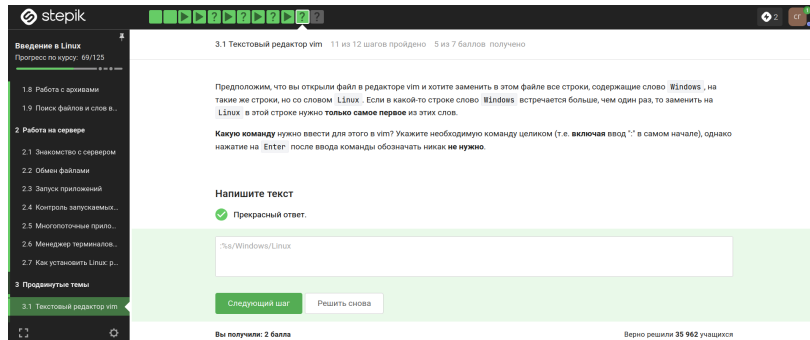


Рисунок 2.4: image4

В режиме выделения открывается из нормального режима по нажатию «v», горит надпись –VISUAL– или –ВИЗУАЛЬНЫЙ РЕЖИМ–, можно использовать команды d(удаление), y(копирование), перемещение (рис. 2.5).

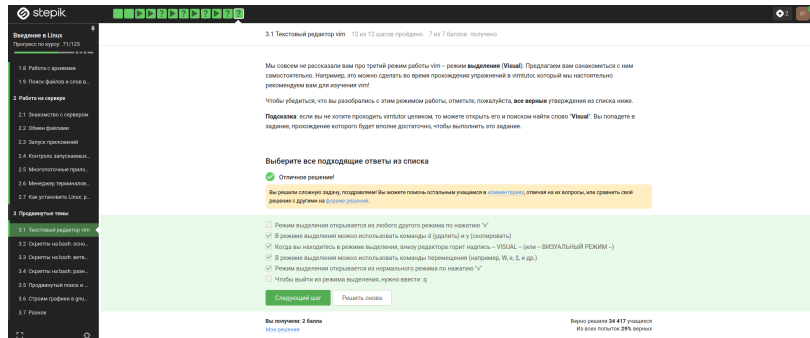


Рисунок 2.5: image5

Только из набора C, так как каждая оболочка имеет свой буфер (при выходе записывается в файл истории) (рис. 2.6).

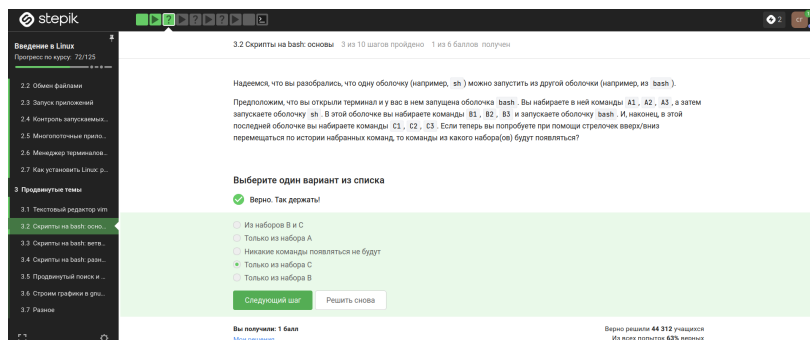


Рисунок 2.6: image6

Файл создается в директории `/home/bi/`, а после мы переходим дальше (рис. 2.7).

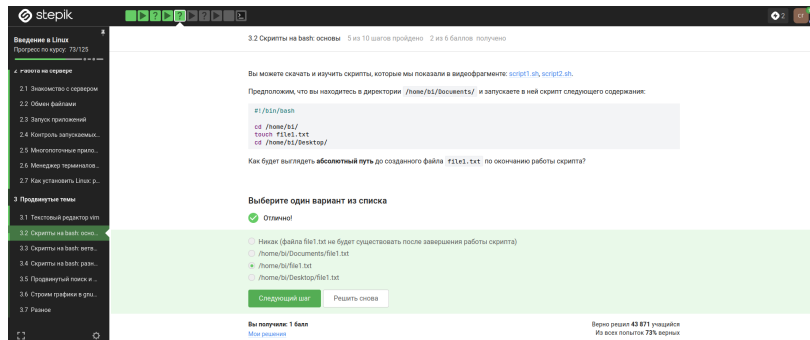


Рисунок 2.7: image7

Имя не может содержать в себе специальные символы и пробелы, а также не может начинаться с цифры (рис. 2.8).

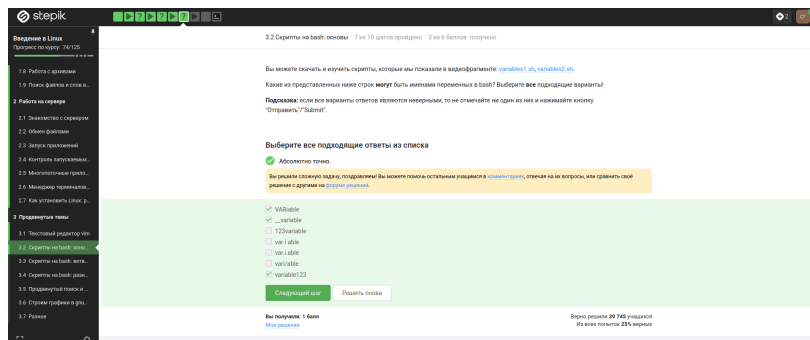


Рисунок 2.8: image8

Задаем переменные `p1` и `p2` и с помощью `echo` печатаем строки с аргументами (рис. 2.9).

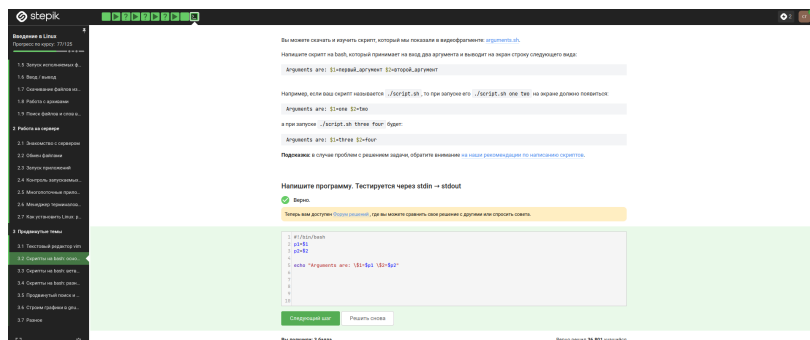


Рисунок 2.9: image9

\$# — количество аргументов, ! — логическое отрицание (НЕ), -s — проверка файла на существование и его размер больше нуля (не пустой), \$0 — имя самого скрипта, -z — проверка длины строки равной нулю, “ ” — пустая строка, -e — проверка файла на существование, -n — проверка длины строки не равной нулю, \$1 — первый аргумент, переданный скрипту (рис. 2.10, рис. 2.11).

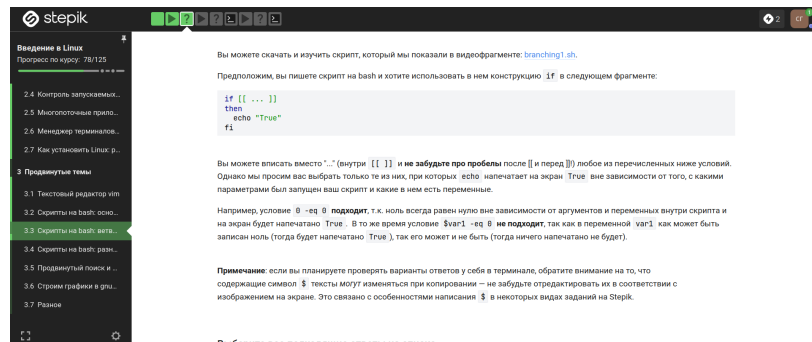


Рисунок 2.10: image10

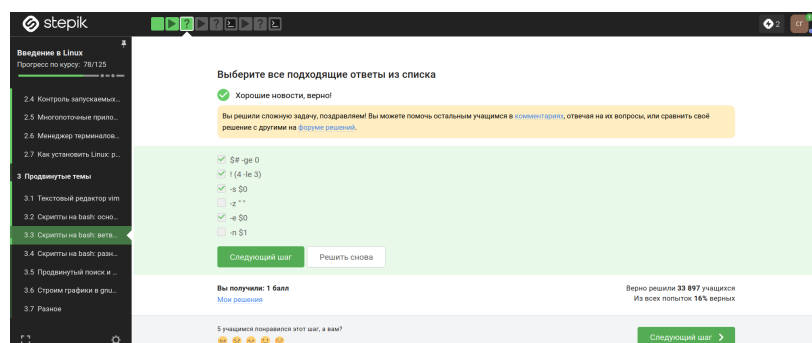


Рисунок 2.11: image11

-gt - больше, -lt - меньше, -eq - равно. То есть выведет four два раза (рис. 2.12).

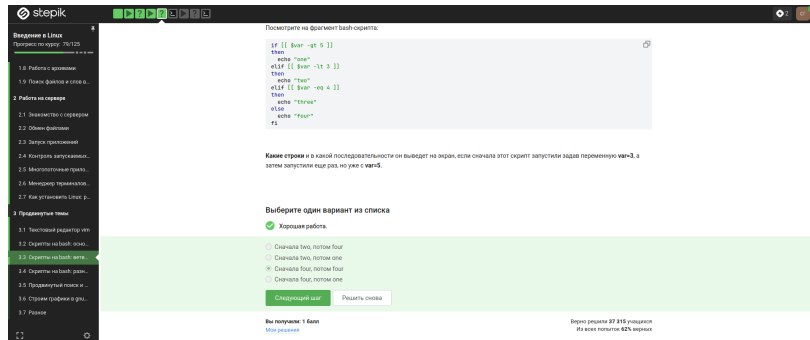


Рисунок 2.12: image12

Задаем переменную, проверка на условия, вывод при определенном условии (рис. 2.13, рис. 2.14).

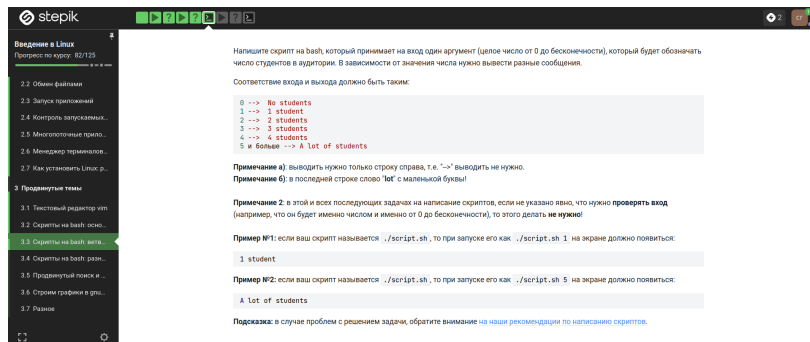


Рисунок 2.13: image13

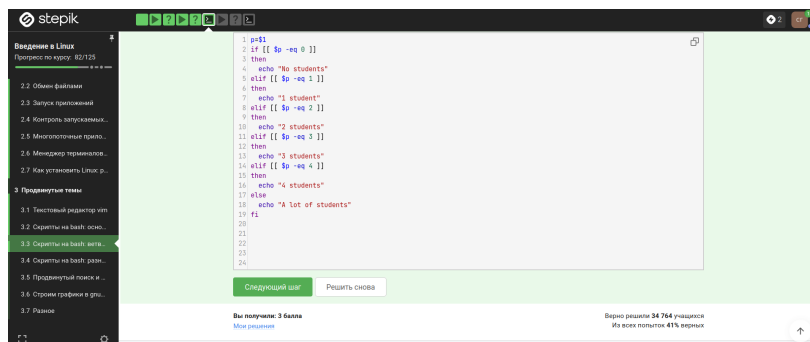
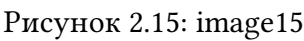


Рисунок 2.14: image14

(Start), $a > c$ нет (Finish), (Start), $\langle \rangle > c$ нет (Finish), (Start), $b > c$ нет (Finish), (Start), $\langle \rangle > c$ нет (Finish), (Start), $c_d > c$ (рис. 2.15).



stepik

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

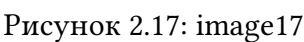
518

519

520

521

Рисунок 2.16: image16



Команда должна быть с `let` и если выражение с пробелами, то должно быть заключено в скобки (рис. 2.18).

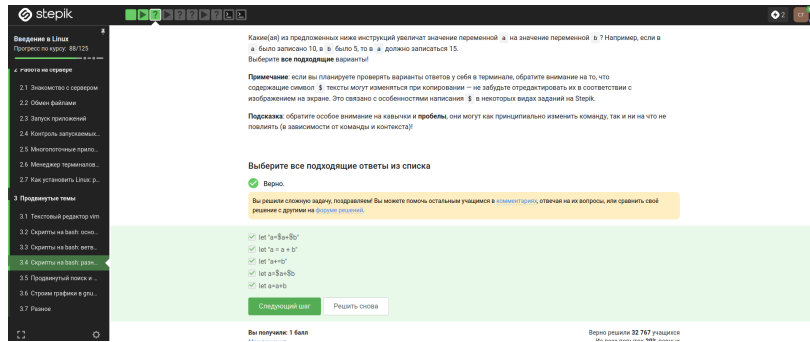


Рисунок 2.18: image18

Команда `pwd` выводит путь директории, в которой сейчас находишься (рис. 2.19).

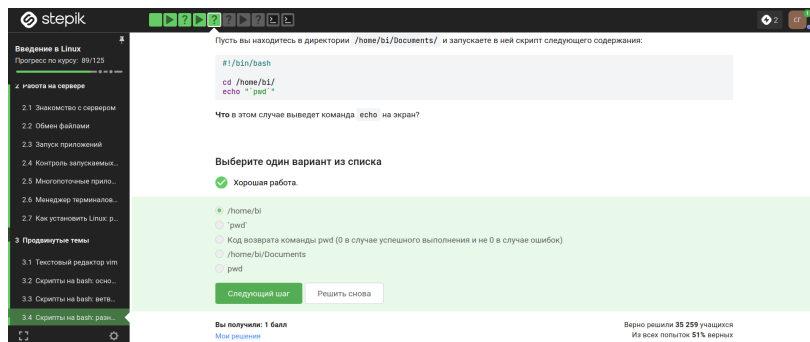


Рисунок 2.19: image19

При `program`` выполняется вывод в терминал и нужно настроить вывод в файл (рис. 2.20).

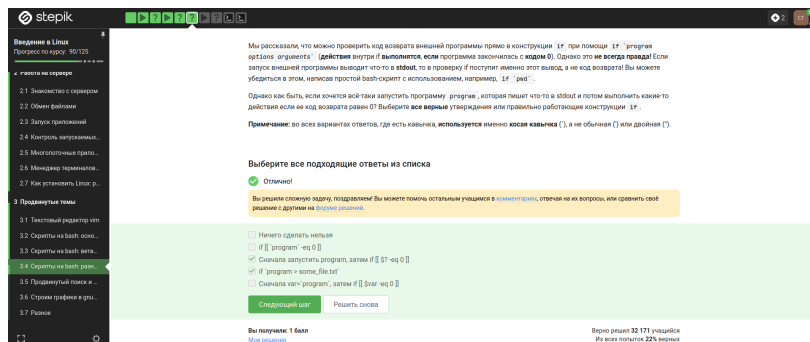
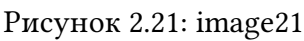


Рисунок 2.20: image20

Первая переменная локальная, вторая - прибавление к сумме числа умноженно-го на 2 (рис. 2.21).



The screenshot shows a video player with a terminal window. The terminal output is as follows:

```

120: greatest common divisor, GCD) двух чисел.
При запуске мы считаем не только наибольший общий делитель, а также все другие натуральные числа, которые делят это число.
Используем функцию gcd и передаем ей два параметра: 120 и 35 (примеры в комментарии). После работы функция gcd считывает из НОД и выводит на экран в строке «наибольший общий делитель»: например, для чисел 120 и 35 это будет «НОД: 5».
Этого этого счетчик будет входить в список «общих делителей натуральных чисел». Если в какой-то момент работы пользователь введет вместо этого
пустую строку, то значение на экран будет bye и закончится свое действие.

Вычисление НОД успешно реализуется с помощью алгоритма Евклида. Мы можем написать функцию gcd, которая принимает на
вход два аргумента (числовых или int). Если аргументы равны, то мы нашли НОД — он равен int (или int), нулю выводить
соответствующие сообщения на экран (см. выше). Иначе можно написать функцию, которая выводит int и int на экран.
Функция gcd, которая принимает на вход два аргумента (числовых или int), и возвращает значение int и int на экран.
Запускаем функцию gcd с параметрами 120 и 35 и смотрим результат.

Пример корректной работы скрипта:

./gcd.sh
120 35
НОД: 5
5
НОД: 35
35
bye

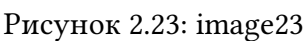
Примечание: в выводе функции из себя самой нет ничего страшного или нежелательного, т.к. само вызовы gcd — это просто вызовы
функции gcd.

Примечание 2: для завершения работы функции в произвольных местах, можно использовать инструкцию return (все инструкции
функции после return будут выполняться в обратном порядке). Это означает, что команда завершит только функцию, а не выполнение всего
скрипта целиком. Скрипт в данном случае можно написать и без использования return.

Переходим к следующему модулю и начинаем изучать основы языка программирования Go и основы работы с файлами.

```

Рисунок 2.22: image22



Напишем калькулятор на `bash`. Выполняться будет таким образом: ввод двух чисел, что с ними нужно сделать, вывод результата (рис. 2.24, рис. 2.25).



Рисунок 2.24: image24

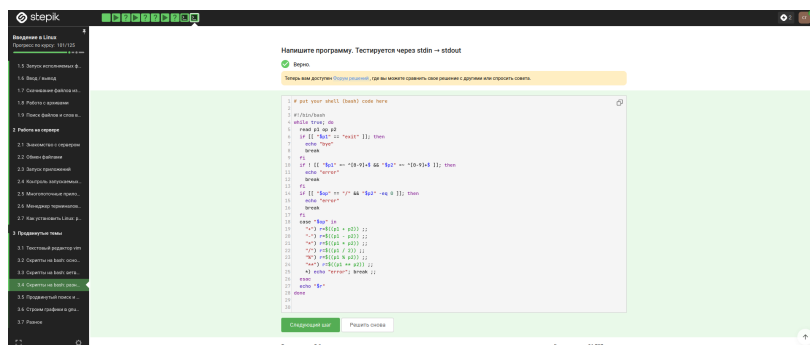


Рисунок 2.25: image25

команда выполнит поиск без учета регистра (-iname) и с неизвестным количеством символов после слова (так как *) (рис. 2.26).

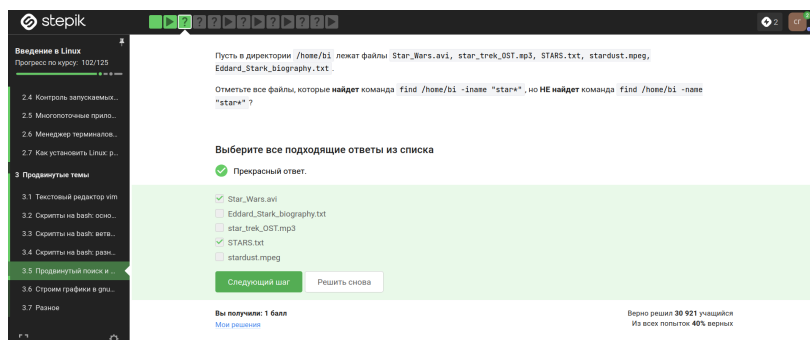


Рисунок 2.26: image26

-path требует совпадения с шаблоном во всём пути, а -name — только в последнем компоненте (имени файла) (рис. 2.27).

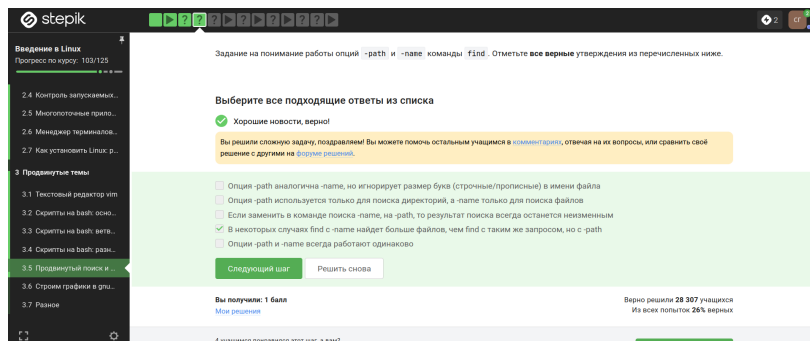


Рисунок 2.27: image27

Считывание выглядит следующим образом: /home/bi, /home/bi/dir1, /home/bi/dir1/dir2, тогда ответ - все кроме file3 (рис. 2.28).

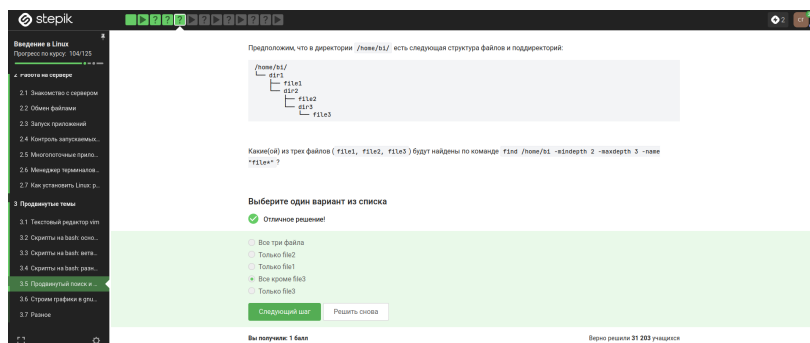


Рисунок 2.28: image28

В результате все четыре команды выведут один и тот же набор строк (весь файл), потому что строки перекрываются и не добавляются новые строки — просто дублирует уже существующие, но итоговое уникальное содержимое (рис. 2.29).

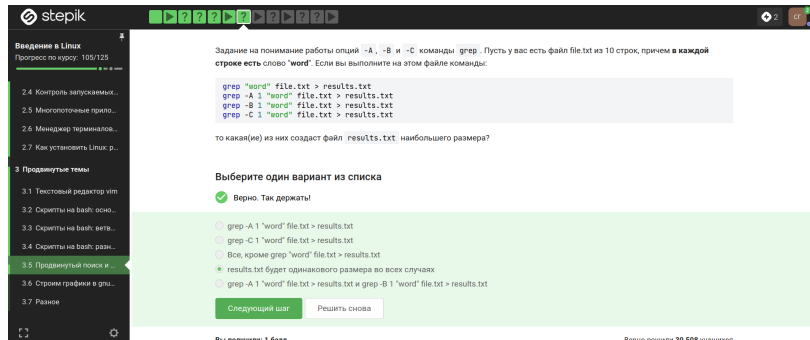


Рисунок 2.29: image29

Все строки заканчиваются на buntu. Перед buntu стоит u или U, а перед ними — ноль или один символ из набора (или нет символа) (рис. 2.30)

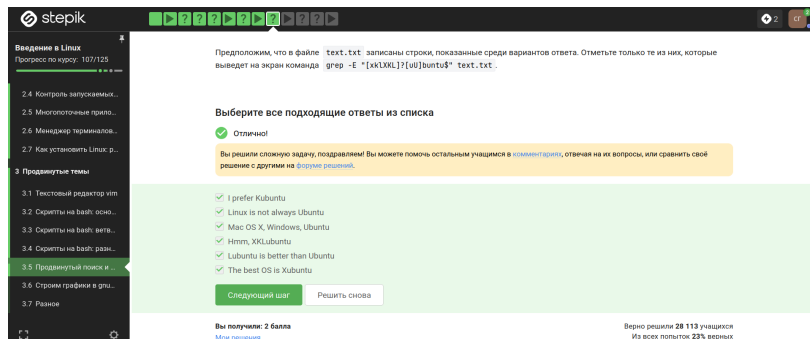


Рисунок 2.30: image30

Каждая строчка будет выведена 2 раза (рис. 2.31).

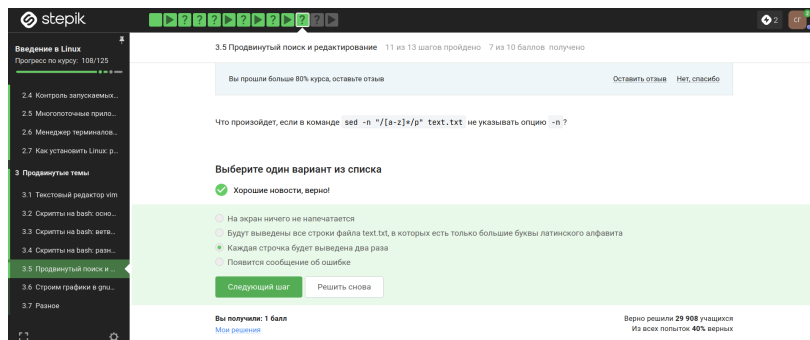


Рисунок 2.31: image31

Ищет последовательности из двух и более заглавных латинских букв и заменяет каждую такую последовательность на слово abbreviation (рис. 2.32, рис. 2.33).

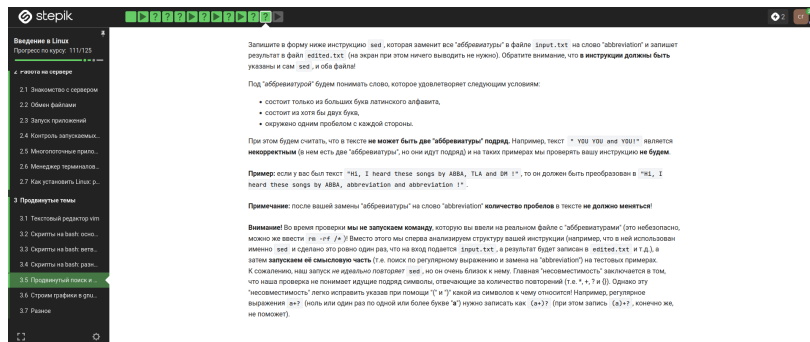


Рисунок 2.32: image32

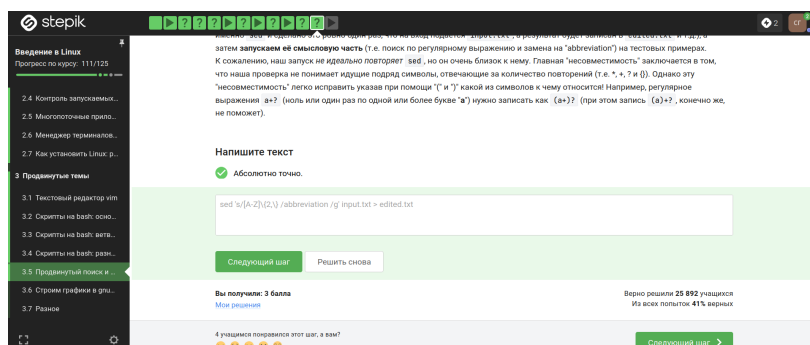


Рисунок 2.33: image33

-p, -persist (рис. 2.34).

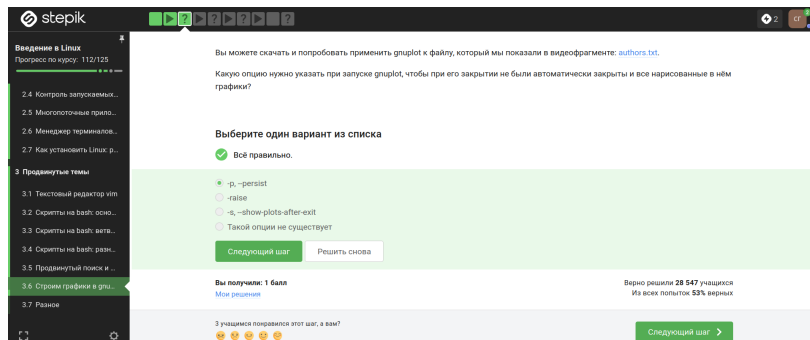


Рисунок 2.34: image34

Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена) (рис. 2.35).

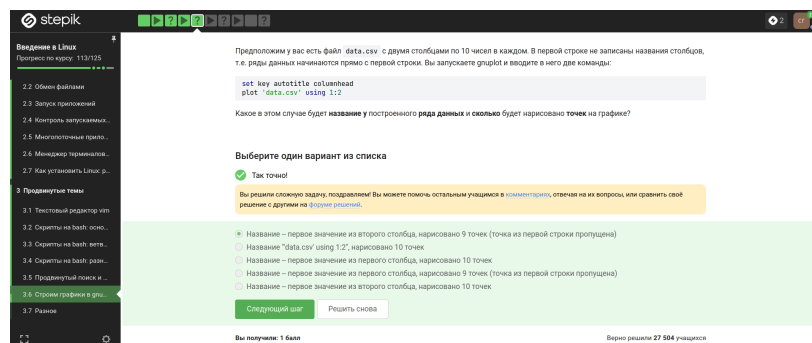


Рисунок 2.35: image35

Команда установки подписей, подпись, пробел, переменная с координатой, запятая. Повтор - соответствует числу переменных (рис. 2.36).

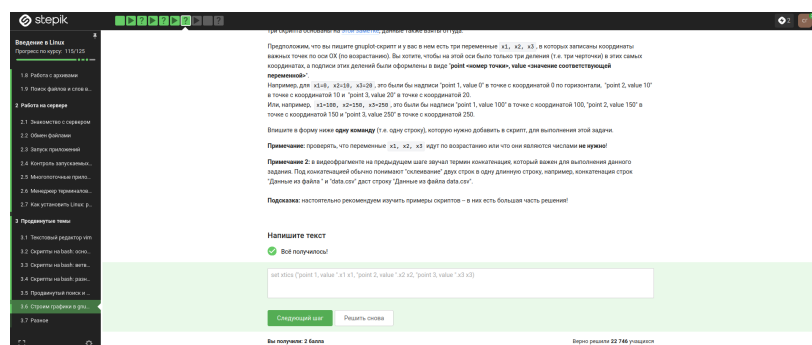


Рисунок 2.36: image36

$b=b+1$ - считает шаги анимации (каждый проход увеличивает счётчик на 1). $zrot=(zrot+350)\%360$ Увеличивает угол поворота на 350 градусов, затем берёт остаток от деления на 360 (чтобы угол всегда был в пределах 0–359). Каждый раз угол меняется на 350° , то есть на следующий кадр картинка поворачивается на -10° (поскольку $+350^\circ$ эквивалентно -10° по модулю 360). (рис. 2.37).

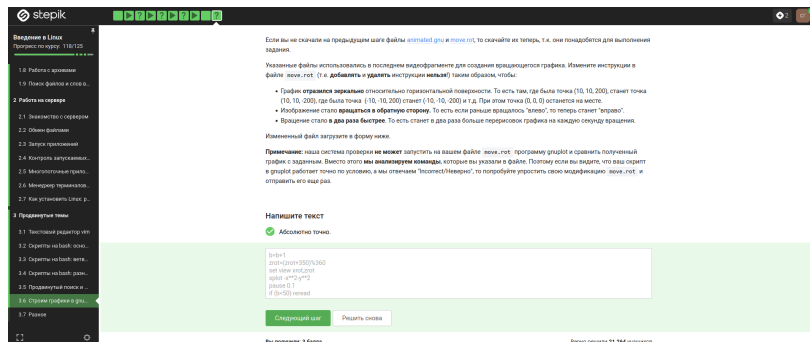


Рисунок 2.37: image37

r - чтение; w - запись; x - выполнение; u - владелец файла; g - группа файла; o - все остальные пользователи; 0 - никаких прав; поэтому `chmod a+wx file.txt`; `chmod o-wx file.txt`; `chmod g-x file.txt` и `chmod u+wx file.txt`; `chmod g+w file.txt` (рис. 2.38).

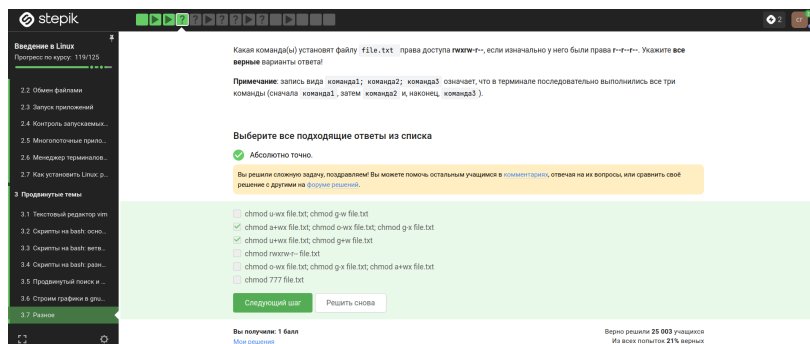


Рисунок 2.38: image38

`sudo chmod 0+w dir`, где 0 — это спец. запись, означающая «не менять права на чтение/исполнение», а +w — добавить право записи для всех (т.к. цифра 0). После этого у dir появляется w для группы (и для остальных), и user из group сможет создать файл. `sudo chown user:group dir`, здесь меняет владельца на user, группу на group. Теперь user (владелец) имеет полные права (rwx) и может писать в директорию (рис. 2.39, рис. 2.40).

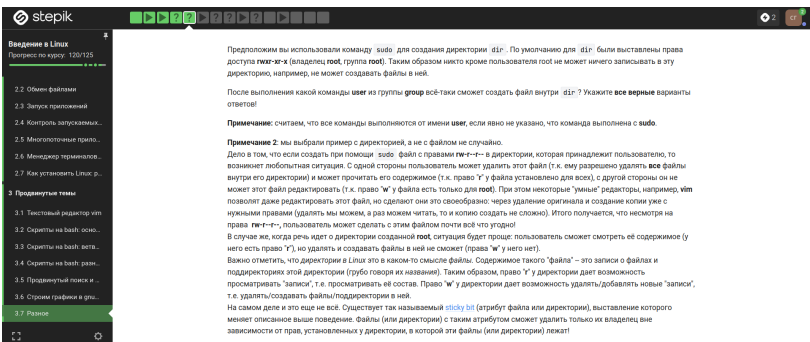


Рисунок 2.39: image39

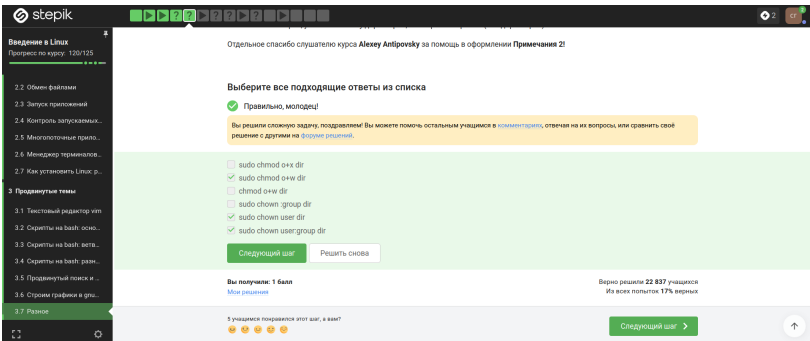


Рисунок 2.40: image40

Количество строк или символов, длину самой длинной строки (рис. 2.41).

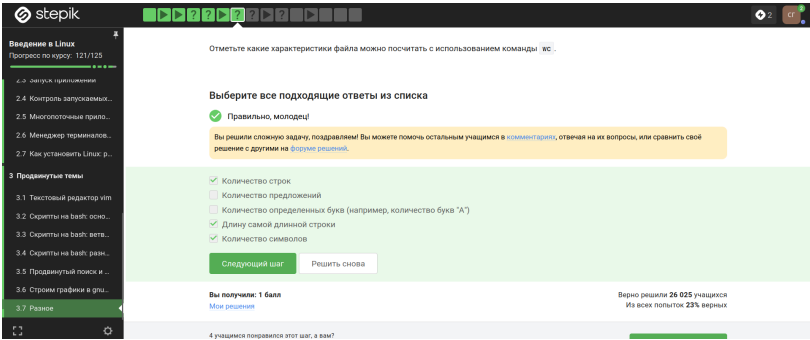


Рисунок 2.41: image41

Команда `du -h -s`, `du` — утилита для оценки использования дискового пространства. `-h` (human-readable) — выводит размер в удобном формате (например, 2.4М вместо 2457600 байт). `-s` (summary) — показывает общий размер текущей директории, не перечисляя каждый файл/поддиректорию (рис. 2.42).

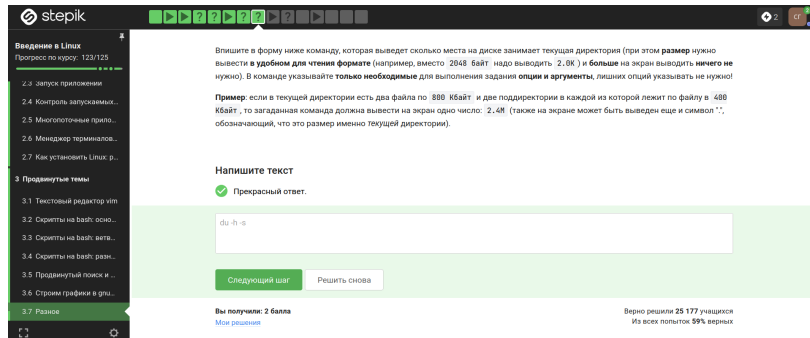


Рисунок 2.42: image42

Эта максимально короткая команда для создания поддиректорий (рис. 2.43).

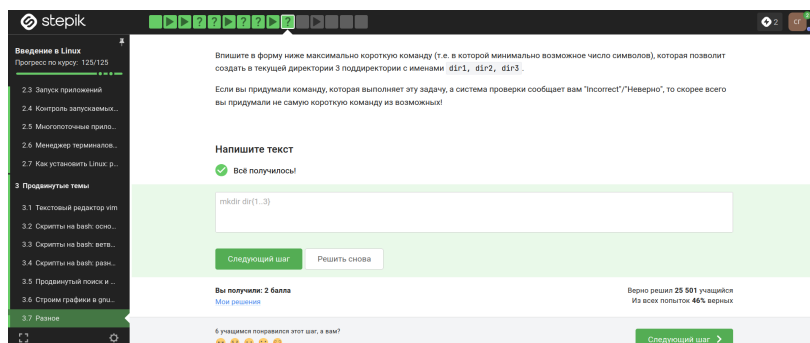


Рисунок 2.43: image43

Сертификат (рис. 2.44, рис. 2.45).

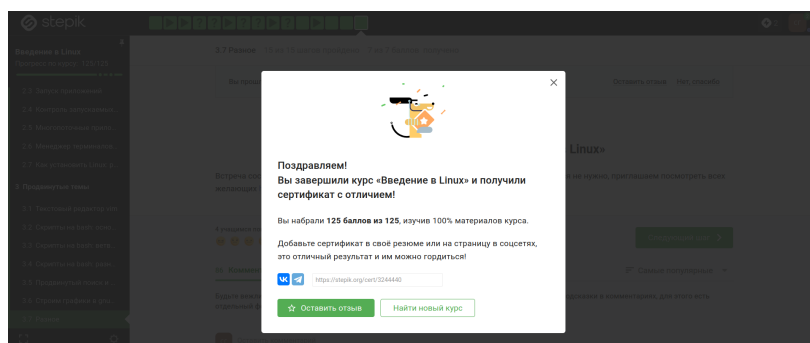


Рисунок 2.44: image44

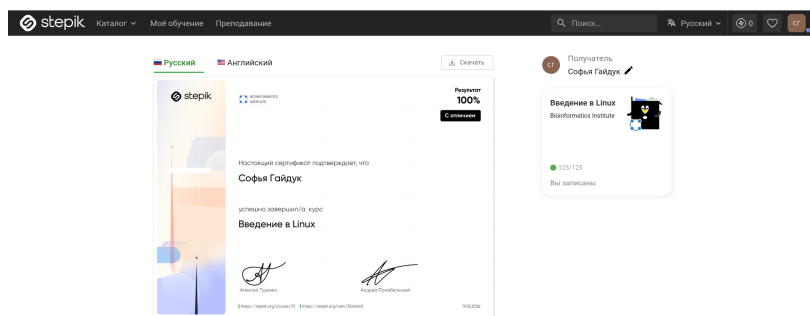


Рисунок 2.45: image45

3 Выводы

Изучили операционную систему Linux. Прошли 3 этап внешнего курса. Полностью прошли, изучили и окончили внешний курс

Список литературы

1.Kulyabov. Прохождение внешнего курса. Часть 3. RUDN